

```
void main()
{
    int a = 10;
    int b = 20;
    IO.println("a == b : " + (a == b) ); //F
    IO.println("a != b : " + (a != b) ); //T
    IO.println("a > b : " + (a > b) ); //F
    IO.println("a < b : " + (a < b) ); //T
    IO.println("b >= a : " + (b >= a) ); //T
    IO.println("b <= a : " + (b <= a) ); //F
}
```

CONTROL STATEMENTS IN JAVA:

In Java, Basically A Java program executes from **top to bottom** but by using control statements we can **control the flow of execution** of a java program.

It helps to decide which code should run, when it should run, and how many times it should run.

Control statements fall into three main categories:

- 1.Selection OR Decision-Making Statements
 - a) if statement
 - b) if-else statement
 - c) else-if statement
 - d) Nested if
 - e) switch statement (JDK 17V with yield as a return value)
- 2.Iteration OR Looping Statements
 - a) do-while loop
 - b) while loop
 - c) for loop
 - d) for each loop (Enhanced for loop)
- 3.Jumping Or Branching statement
 - a) break
 - b) continue
 - c) return
 - d) yield

If condition:

It is decision making statement.
It is used to test a boolean expression.
The expression must **return boolean** type.
It allows us to **execute a block of code only when a given condition is true**.
If the condition returns to **false**, the control skips the if block and moves to the next statement.
The **if statement** is used when we want to execute a block of code **only if a condition is true**, and **does not perform any action, if condition is false**.

Syntax:

```
if(condition)
{
    // statements executed when condition is true
}
```

Programs: [Real time examples]

- a) Displaying a low battery warning

```
void main()
{
    int batteryPercentage = Integer.parseInt(IO.readLine("Enter battery percentage :"));

    if(batteryPercentage < 20)
    {
        IO.println("Low Battery");
    }

    IO.println("Program completed");
}
```

- b) Password Length Validation

* String class has provided a predefined non static method called **length()** through which we can find out the length of the given String.

```
public int length();

Example :
String str = "java technology";
int length = str.length();
IO.println(length); //15
```

Password.java

```
void main()
{
    String password = IO.readLine("Enter your password :");

    if(password.length() < 8)
    {
        IO.println("Password is weak because length is less than 8");
    }
}
```

- c) Internet Connectivity Warning

```
void main()
{
    boolean isConnected = false;

    if(!isConnected)
    {
        IO.println("Not connected through internet");
    }
}
```

Assignemnts :

- Speed Limit Alert (Vehicle System)
- Bank Minimum Balance Alert
- Invalid roll number

When to use if-else

The if-else statement is used when one action must be performed if the condition is true and a different action must be performed if the condition is false.

Syntax:

```
if(condition)
{
    // statements executed when condition is true
}
else
{
    // statements executed when condition is false
}
```

- a) Number is even or odd

```
void main()
{
    int num = Integer.parseInt(IO.readLine("Enter a number :"));

    if(num %2==0)
    {
        IO.println(num + " is even");
    }
    else
    {
        IO.println(num + " is odd");
    }
}
```

- Train berth available or not

```
void main()
{
    int available = 1;
    int wanted = Integer.parseInt(IO.readLine("Enter the number of berth :"));

    if(available >= wanted)
    {
        IO.println("Berth is available");
    }
    else
    {
        IO.println("Berth is not available");
    }
}
```

When to use else if

If, we want to check **multiple conditions** one after another and wants to execute only one matching block among all the conditional blocks then we should use else-if.

Syntax:

```
if(condition1)
{
    // statements executed when condition1 is true
}
else if(condition2)
{
    // statements executed when condition2 is true
}
else if(condition3)
{
    // statements executed when condition3 is true
}
else
{
    // statements executed when all conditions are false
}
```

- a) Student grading system [>=90 grade A, >=75 grade B, >=60 grade c else fail]

```
void main()
{
    int marks = Integer.parseInt(IO.readLine("Enter your Marks :"));

    if(marks >=90)
    {
        IO.println("Grade A");
    }
    else if(marks >=75)
    {
        IO.println("Grade B");
    }
    else if(marks >=60)
    {
        IO.println("Grade C");
    }
    else
    {
        IO.println("Fail");
    }
}
```

- Traffic Signal System [String signal = "YELLOW";]

How to compare two String value :

In order to compare String values we can use the following two method given by String class :

- a) public boolean **equals**(Object obj) :

* It is an overridden method of String class, used to compare two String objects based on the **content**.
* It is case sensitive method so, If two string objects are having same content then It will return true otherwise false.

Example :
String str = "java";
boolean isEqual = str.equals("java");

- b) public boolean equalsIgnoreCase(String str) :

* Used to compare two String object based on the content by ignoring the case, that means a and A both are same

```
void main()
{
    String signal = IO.readLine("Enter Signal Color :");

    if(signal.equalsIgnoreCase("Red"))
    {
        IO.println("STOP");
    }
    else if(signal.equalsIgnoreCase("Yellow"))
    {
        IO.println("WAIT");
    }
    else
    {
        IO.println("GO");
    }
}
```

Note : Never compare String objects by using == operator because Strings are immutable